
BCSE102L

Structured and Object – Oriented Programming

Dr. Pramod Kumar Maurya

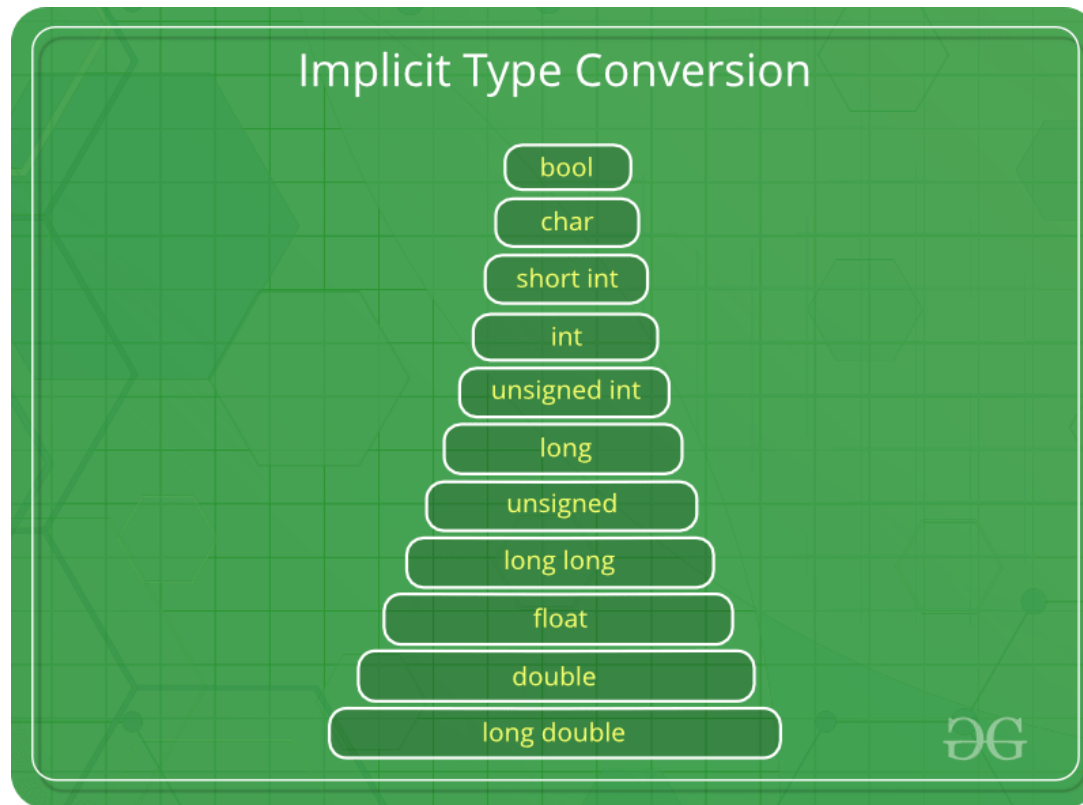
School of Computer Science and Engineering
VIT Vellore

Type Conversion in C

- Type conversion in C is the process of converting one data type to another.
- The type conversion is only performed to those data types where conversion is possible.
- Type conversion is performed by a compiler.
- In type conversion, the destination data type can't be smaller than the source data type.
- ***There are two types of Conversion:***
 - **Implicit Type Conversion**
 - **Explicit Type Conversion**

Implicit Type Conversion

- Also known as 'automatic type conversion'.
 - Done by the compiler on its own, without any external trigger from the user.



Example of Type Implicit Conversion

```
// An example of implicit conversion
#include <stdio.h>
int main()
{
    int x = 10; // integer x
    char y = 'a'; // character c

    // y implicitly converted to int. ASCII
    // value of 'a' is 97
    x = x + y;

    // x is implicitly converted to float
    float z = x + 1.0;

    printf("x = %d, z = %f", x, z);
    return 0;
}
```

- **Output**

- $x = 107, z = 108.000000$

Explicit Type Conversion

- This process is also called type casting and it is user-defined.
- Here the user can typecast the result to make it of a particular data type. The syntax in C Programming:

(type) expression

- Type indicated the data type to which the final result is converted.

Example of Type Explicit Conversion

```
// C program to demonstrate explicit
type casting
#include<stdio.h>

int main()
{
    double x = 1.2;

    // Explicit conversion from double
    to int
    int sum = (int)x + 1;

    printf("sum = %d", sum);

    return 0;
}
```

- **Output**
- `sum = 2`

Control Structures

- A control structure refers to the way in which the programmer specifies the order of execution of the statements.
- The control structures are classified into
 - Sequential: In this approach, all the statements are executed in the same order as it is written.
 - Selectional: In this approach, based on some conditions, different set of statements are executed.
 - Iterational or Repetition: In this approach, a set of statements are executed repeatedly

Selection of Control Structures

- There are two categories of selectional control structures
- Conditional control statements
 - if statement
 - switch statement
- Unconditional control statements
 - goto statement

Conditional Statement in C

- **Conditional Statements in C** programming are used to make decisions based on the conditions.
 - Conditional statements execute sequentially when there is no condition around the statements.
 - If you put some condition for a block of statements, the execution flow may change based on the result evaluated by the condition.
 - This process is called decision making in 'C.'
-

Decision/Selection statements

- if
- if else
- nested if
- if else if
- switch case

Syntax of if Statement Two Alternatives

Form 1:

```
if ( condition )  
    statement ; // True
```

Eg:

```
if (x > 0.0)  
    pos_prod = pos_prod * x;
```

Syntax of if Statement Two Alternatives

Form 2:

if (condition)

statement ; // True

else

statement ; // False

Example

```
if (x >= 0.0)
```

```
    printf("positive\n");
```

```
else
```

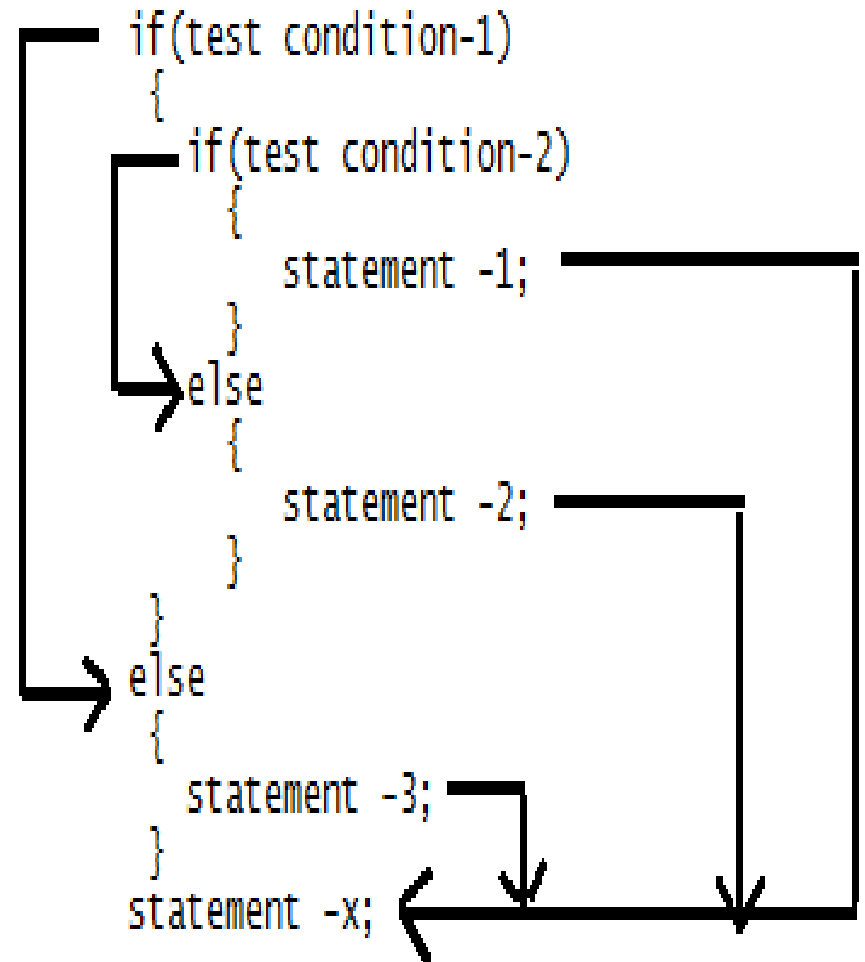
```
    printf("negative\n");
```

Write a program to check whether the given year is a leap year or not

```
int main()
{
    int year;
    scanf("%d", &year);
    if(((year%4==0) && ((year%400==0) || (year%100!=0)))
        printf(" the given year is a leap Year");
    else
        printf(" the given year is not a leap year");
    return 0;
}
```

If Statement in C

```
if (test expression1) {  
    // statement(s)  
}  
else if(test expression2) {  
    // statement(s)  
}  
else if (test expression3) {  
    // statement(s)  
}  
.  
.  
else {  
    // statement(s)  
}
```



```
int main()
{
    int a=5,b=4,c=6;
    printf ("a=%d b=%d c=%d \n",a,b,c);
    if (a>b)
    {
        if (a>c)
            printf("The biggest is = %d \n",a);
        else
            printf("The biggest is =%d",c);
    }
    else
    {
        if (b>c)
            printf("The biggest is = %d \n",b);
        else
            printf("The biggest is = %d \n",c);
    }
    return 0;
}
```

Short Circuit Evaluation

- Stopping evaluation of a logical expression as soon as its value can be determined is called short-circuit evaluation
 - **Second part of '&&'** does not gets evaluated when first part is evaluated as False
 - **Second part of '||'** does not gets evaluated when first part is evaluated as true
-

Logical Assignment

- `in_range = (n > -10 && n < 10);`
- `is_letter = ('A' <= ch && ch <= 'Z') || ('a' <= ch && ch <= 'z');`

Class of the Ship

- Each ship serial number begins with a letter indicating the class of the ship. Write a program that reads a ship's first character of serial number and displays the class of the ship.

Class ID	Ship Class
B or b	Battleship
C or c	Cruiser
D or d	Destroyer
F or f	Frigate

Nested if Statements

```
#include<stdio.h>
void main()
{
    char c;
    scanf("%c",&c);
    if ((c=='b')||(c=='B'))
        printf("Battleship");
    else if ((c=='c')||(c=='C'))
        printf("Cruiser");
    else if ((c=='d')||(c=='D'))
        printf("Destroyer");
    else if ((c=='f')||(c=='F'))
        printf("Frigate");
}
```

Switch Case statement

- The Switch statement is a selectional control structure that selects a
- choice from the set of available choices.
- It is similar to if statement but not a replacement for if statement.

Syntax of Switch Statement

```
Switch(expression){  
  case integer or character constant-1: Statement(s) ;  
    break;  
  case integer or character constant-2: Statement(s) ;  
    break;  
  case integer or character constant-n: Statement(s) ;  
    break;  
  default: Statement(s) ;  
    break;  
}
```

```
#include<stdio.h>
void main()
{
char c;
scanf("%c",&c);
switch(c)
{
case 'b':
case 'B':
printf("Battleship");
case 'c':
case 'C':
printf("Cruiser");
case 'd':
case 'D':
printf("Destroyer");
case 'f':
case 'F':
printf("Frigate");
default:
printf("No match");
}
}
```

Output

BattleshipCruiserDestroyerFrigateNo
match

When input is b

```
#include<stdio.h>
void main()
{
char c;
scanf("%c",&c);
switch(c)
{
case 'b':
case 'B':
printf("Battleship");
break;
case 'c':
case 'C':
printf("Cruiser");
break;
case 'd':
case 'D':
printf("Destroyer");
break;
case 'f':
case 'F':
printf("Frigate");
}
}
```

Practice problem

Make use of the Switch Case Structure to write a program and print the number of days in a month by reading the year and month(number).

Note : Make use of the year value to compute the days in a month when the given year is a leap year.

Program: Calculating a Broker's Commission

- When stocks are sold or purchased through a broker, the broker's commission often depends upon the value of the stocks traded.
- Suppose that a broker charges the amounts shown in the following table:

<i>Transaction size</i>	<i>Commission rate</i>
Under \$2,500	\$30 + 1.7%
\$2,500–\$6,250	\$56 + 0.66%
\$6,250–\$20,000	\$76 + 0.34%
\$20,000–\$50,000	\$100 + 0.22%
\$50,000–\$500,000	\$155 + 0.11%
Over \$500,000	\$255 + 0.09%

- The minimum charge is \$39.

Program: Calculating a Broker's Commission

- The `broker.c` program asks the user to enter the amount of the trade, then displays the amount of the commission:

```
Enter value of trade: 30000
```

```
Commission: $166.00
```

- The heart of the program is a cascaded `if` statement that determines which range the trade falls into.
-

```
/* Calculates a broker's commission */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float commission, value;
```

```
    printf("Enter value of trade: ");
```

```
    scanf("%f", &value);
```

```
    if (value < 2500.00f)
```

```
        commission = 30.00f + .017f * value;
```

```
    else if (value < 6250.00f)
```

```
        commission = 56.00f + .0066f * value;
```

```
    else if (value < 20000.00f)
```

```
        commission = 76.00f + .0034f * value;
```

```
    else if (value < 50000.00f)
```

```
        commission = 100.00f + .0022f * value;
```

```
    else if (value < 500000.00f)
```

```
        commission = 155.00f + .0011f * value;
```

```
    else
```

```
        commission = 255.00f + .0009f * value;
```

```
    if (commission < 39.00f)
```

```
        commission = 39.00f;
```

```
    printf("Commission: $%.2f\n", commission);
```

```
    return 0;
```

```
}
```

Iterative Control Structures

- Iterative or repetitive control structures are used to repeat certain statements for a specified number of times.
- The statements are executed as long as the condition is true They are also called as loop control structures
- The three looping structures are
 - while
 - do while
 - for

While Statement in **C**

```
while (expression)
{
    // execute statements
}
```

```
#include<stdio.h>
```

```
// Program to illustrate while .. loop
```

```
main()
```

```
{
```

```
    int i,a,n,sum=0;
```

```
    float avg;
```

```
    printf("Enter the number of integers");
```

```
    scanf("%d",&n);
```

```
    printf(" Enter %d integers",n);
```

```
    i=1;
```

```
    while (i<=n)
```

```
{
```

```
        printf("\n %d :",i);
```

```
        scanf("%d",&a);
```

```
        sum+=a;
```

```
        i++;
```

```
    }
```

```
    avg=(float)sum/n;
```

```
    printf(" The average of %d numbers is %f",n,avg);
```

```
}
```

For Statement in C

The *initialization*
is executed once
before the loop begins

The *statement* is
executed until the
condition becomes false

```
for ( initialization ; condition ; increment )  
{  
    statement;  
}
```

The *increment* portion is
executed at the end of each
iteration

```
#include<stdio.h>
```

```
// Program to illustrate for loop
```

```
main()
```

```
{
```

```
    int l,a,n,sum=0;
```

```
    float avg;
```

```
    printf("Enter the number of integers");
```

```
    scanf("%d",&n);
```

```
    printf(" Enter %d integers",n);
```

```
    for(i=1; i<=n; i++)
```

```
    {
```

```
        printf("%d :",i);
```

```
        scanf("%d",&a);
```

```
        sum+=a;
```

```
    }
```

```
    avg=(float)sum/n;
```

```
    printf(" The average of %d numbers is %f",n,avg);
```

```
}
```

Do while Loop

- Similar to a while loop, except the fact that it is guaranteed to execute at least one time
- Syntax :

do

{

statement(s);

} while(condition);

Condition is checked at the end of execution

```
#include<stdio.h>                // Program to illustrate do..while loop
main()
{
    int l,a,n,sum=0;
    float avg;
    printf("Enter the number of integers");
    scanf("%d",&n);
    printf(" Enter %d integers",n);
    i=1;
    do
    {
        printf("\n %d :",i);
        scanf("%d",&a);
        sum+=a;
        i++;
    } while(i<=n);
    avg=(float)sum/n;
    printf(" The average of %d numbers is %f",n,avg);
}
```

```
#include<stdio.h>
void main()
{
int i = 0;
for(;i<10;)
{
printf("Hello");
i++;
}
}
```

OUTPUT :

Hello

Hello

Hello

Hello

Hello

Hello

Hello

Hello

Hello

Hello

C break statement

- The break is a keyword in C which is used to bring the program control out of the loop.
- The break statement is used inside loops or switch statement.
- The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops.
- The break statement in C can be used in the following two scenarios:
 - With switch case
 - With loop

```
1.#include<stdio.h>
2.#include<stdlib.h>
3.void main ()
4.{
5.    int i;
6.    for(i = 0; i<10; i++)
7.    {
8.        printf("%d ",i);
9.        if(i == 5)
10.            break;
11.    }
12.    printf("came outside of loop i = %d",i);
13.
14.}
```

Output

```
0 1 2 3 4 5 came outside of loop i = 5
```

C continue statement

- The continue statement skips some lines of code inside the loop and continues with the next iteration.
- It is mainly used for a condition so that we can skip some code for a particular condition.

```
//loop statements
```

```
continue;
```

```
//some lines of the code which is to be skipped
```

Continue statement example 1

```
#include<stdio.h>
int main(){
int i=1;//initializing a local variable
//starting a loop from 1 to 10
for(i=1;i<=10;i++){
if(i==5){
//if value of i is equal to 5, it will continue the loop
continue;
}
printf("%d \n",i);
} //end of for loop
return 0;
}
```

Output

```
1  
2  
3  
4  
6  
7  
8  
9  
10
```

C goto statement

- The goto statement is known as jump statement in C.
- As the name suggests, goto is used to transfer the program control to a predefined label.
- The goto statement can be used to repeat some part of the code for a particular condition.
- Syntax:
label:
//some part of the code;
goto label;

goto example

```
1.#include <stdio.h>
2.int main()
3.{
4.  int num,i=1;
5.  printf("Enter the number whose table you want
to print?");
6.  scanf("%d",&num);
7.  table:
8.  printf("%d x %d = %d\n",num,i,num*i);
9.  i++;
10. if(i<=10)
11.  goto table;
12.}
```

Output:

```
Enter the number whose table you want to print?10
```

```
10 x 1 = 10
```

```
10 x 2 = 20
```

```
10 x 3 = 30
```

```
10 x 4 = 40
```

```
10 x 5 = 50
```

```
10 x 6 = 60
```

```
10 x 7 = 70
```

```
10 x 8 = 80
```

```
10 x 9 = 90
```

```
10 x 10 = 100
```